Python Imports, Packages, sys.path, pytest, and uv

A complete guide for the Smart File Organizer project

This guide explains modules, packages, sys.path, -m, PYTHONPATH, pytest, uv, and pyproject.toml.

It includes current-project behavior, direct-execution cases, and the packaged-project workflow.

Quick model

```
script or module -> sys.path -> import search -> selected module
```

Key rule: Python does not search the whole repository automatically.
It searches directories listed in sys.path, including locations added by PYTHONPATH,
pytest configuration, or an installed package.

Contents overview

1. Current project and original import error
2. Modules, packages, and sys.path
3. Direct execution, -m, pytest, and PYTHONPATH
4. __pycache__, pyproject.toml, uv, and editable installs
5. src layout, package imports, and working-directory paths
6. Comparisons, command reference, and troubleshooting

Current project commands

```
uv run python -m pytest test est_rule_engine.py
uv run pytest test est_rule_engine.py
$env:PYTHONPATH='.'; uv run python test est_rule_engine.py
```

regular package. Modern Python may allow the command without it as a namespace package, but adding it improves compatibility with tools and makes the intent explicit.

Running -m from the wrong directory

```
cd "C:\Smart file organizer\test"
uv run python -m test_rule_engine
```

This can find test_rule_engine.py in the current test directory, but it does not necessarily make the parent project root importable. Running the dotted command from the project root is the reliable form:

```
cd "C:\Smart file organizer"
uv run python -m test.test_rule_engine
```

8. pytest commands and why they differ

```
uv run pytest test\test_rule_engine.py
uv run python -m pytest test\test_rule_engine.py
```

The first command runs the pytest executable. The second starts Python and asks it to run the pytest module. The -m form makes Python's module execution and current-directory behavior explicit.

The pytest.ini file in this project contains:

```
[pytest]
pythonpath = .
```

This tells pytest to add the project root to its import path. It affects pytest runs, not direct commands such as uv run python test\test_rule_engine.py.

The recommended current-project command is:

```
uv run python -m pytest test\test_rule_engine.py
```

or, with pytest.ini present:

```
uv run pytest test\test_rule_engine.py
```

A test file should define functions whose names start with test_. Top-level print statements are executed during import/collection but are not collected as tests. That is why pytest can report collected 0 items.

9. The test-specific issues encountered

The test once called item.get_file_name(), but FileItem defines extract_file_name(). Those names must match.

```
print(item.extract_file_name(), "->", engine.get_category(item))
```

The rule engine also returned lowercase other while the test expected Others. The default category should be made consistent, for example:

```
return self.extension_rules.get(extension, "Others")
```

10. What is PYTHONPATH?

PYTHONPATH is an environment variable containing extra directories that Python adds to sys.path. From the project root:

```
$env:PYTHONPATH="C:\Smart file organizer"
```

After that, Python commands in the same PowerShell session can import root modules even when the script is elsewhere. You can set it to the current directory with $env:PYTHONPATH='.' from the project root.

It is session-only unless configured permanently. A persistent user-level setting can be created with:

```
[Environment]::SetEnvironmentVariable(
    "PYTHONPATH",
    "C:\Smart file organizer",
    "User"
```

```
)
```

A new terminal is normally required after changing persistent environment variables. Remove the setting with the same method and a null value.
PYTHONPATH affects imports, not relative file paths. Path('downloads') is resolved relative to the process working directory, not automatically relative to the project root.
A global PYTHONPATH is convenient but can cause collisions between projects that contain similarly named files. Project packaging is generally safer.

11. __pycache__ and .pyc files

Python may compile imported source files into bytecode. These files are stored in __pycache__ directories close to their source files.

```
C:\Smart file organizer\__pycache__\File_item.cpython-311.pyc
C:\Smart file organizer\test\__pycache__\test_rule_engine.cpython-311-pytest-9.0.2.pyc
```

The filename identifies the module, Python implementation, and version. __pycache__ is generated, can be safely deleted, and will be recreated. It should normally not be committed.

```
__pycache__/
*.py[cod]
.pytest_cache/
```

12. What is pyproject.toml?

pyproject.toml is the standard configuration file for a modern Python project. It can declare project metadata, supported Python versions, dependencies, build settings, package discovery, command-line entry points, and tool configuration.
The current project does not have one because uv init has not been run and the project has not yet been configured as an installable package.

```
[project]
name = "smart-file-organizer"
version = "0.1.0"
description = "A command-line tool for organizing files by type"
requires-python = ">=3.10"
dependencies = []

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src/smart_file_organizer"]
```

The distribution/project name may use hyphens, while the import package normally uses underscores: smart-file-organizer versus smart_file_organizer.
uv init creates project metadata, usually including pyproject.toml. It does not automatically move files, create all package imports, or convert loose modules into a package.

13. What does uv run do?

uv run executes a command using the project's managed Python environment. Examples are uv run python main.py and uv run pytest. It does not automatically turn every root-level .py file into an installed package.
Project layout, pyproject.toml, package discovery, and installation still determine what importable package is available.

14. What does uv pip install -e . mean?

```
uv pip install -e .
```

uv is the environment/package manager, pip is the pip-compatible installer interface,

install requests installation, -e means editable, and . means the project in the current directory.
This command is run by the developer or user. Python does not automatically install every directory containing Python files. The project normally needs pyproject.toml so the installer knows the project name, package source location, dependencies, and build backend.
An editable install records a link or configuration in the active environment that points the package name to the source directory. Source edits are immediately used without reinstalling after each change.
Reinstallation may still be needed after changing dependencies, package configuration, package location, or metadata.

15. Packaged project layout

```
C:\Smart file organizer\
|-- pyproject.toml
|-- src\
|   ‘-- smart_file_organizer\
|       |-- __init__.py
|       |-- file_item.py
|       |-- rule_engine.py
|       |-- file_scanner.py
|       |-- file_mover.py
|       |-- move_planner.py
|       |-- organizer.py
|       ‘-- main.py
‘-- tests\
    ‘-- test_rule_engine.py
```

The src layout is common because it prevents accidental imports directly from the repository root. Related files remain separate modules, but they are grouped under one import namespace.
Inside the package, imports can use relative form such as from .file_item import FileItem.
Tests should use the full package path: from smart_file_organizer.file_item import FileItem.

16. Installing and using the package

```
cd "C:\Smart file organizer"
uv init
uv pip install -e .
uv run pytest tests\test_rule_engine.py
```

After installation, Python’s environment knows that smart_file_organizer points to the source package. From another directory, using the same environment, this can work:

```
cd C:\Users\rana
python -c "from smart_file_organizer.file_item import FileItem; print(FileItem)"
```

When using uv from outside the project, explicitly selecting the project can be useful:

```
uv run --project "C:\Smart file organizer" python -c "from smart_file_organizer.file_item import FileItem; print
```

The package import no longer depends on finding file_item.py in the current directory. The environment knows the package location.

17. Imports versus working-directory file paths

Package installation fixes Python imports, but it does not change the process working directory.

```
Organizer("downloads")
Path("downloads")
```

If launched from C:\Smart file organizer, downloads usually means C:\Smart file organizer\downloads. If launched from C:\Users\rana, it may mean C:\Users\rana\downloads. A robust application should accept an explicit directory or resolve paths intentionally.

18. Why packages scale better

Packages provide a stable namespace, reduce dependence on the current directory, avoid collisions with unrelated modules, organize related code, make testing more reliable, separate application code from tests, and make distribution and CLI entry points possible.

```
[project.scripts]
smart-organizer = "smart_file_organizer.main:main"
```

After packaging, users can eventually run smart-organizer instead of python main.py.

19. Complete comparison

```
Current loose-module project:
    from File_item import FileItem
    Depends on sys.path, pytest.ini, PYTHONPATH, or launch location.

Installed package project:
    from smart_file_organizer.file_item import FileItem
    Depends on the package being installed in the active environment.
```

In the current project, direct execution of a root script usually works, direct execution of a subdirectory script may fail, pytest.ini helps pytest, -m from the root helps module execution, and PYTHONPATH can provide a temporary workaround.
In the packaged project, imports work from different directories when the correct environment is active. The package is discovered through the environment rather than by searching the current directory for loose files.

20. Command reference

```
# Run the application from the repository root
uv run python main.py

# Run one test file through pytest
uv run pytest test\test_rule_engine.py

# Equivalent explicit module launch
uv run python -m pytest test\test_rule_engine.py

# Run a test module by dotted module name from the root
uv run python -m test.test_rule_engine

# Run a direct subdirectory script with a temporary import path
$env:PYTHONPATH="C:\Smart file organizer"
uv run python test\test_rule_engine.py

# Inspect import search locations
uv run python -c "import sys; print(*sys.path, sep='\n')"

# Initialize project metadata
uv init

# Install current project in editable mode
uv pip install -e .
```

21. Recommended path for Smart File Organizer

For the current initial phase, keep the existing modules, retain pytest.ini, and run tests through pytest:

```
uv run python -m pytest test\test_rule_engine.py
```

When the project grows, convert it to an installable package: run uv init, create src\smart_file_organizer\, add __init__.py, move application modules, use lowercase conventional filenames, update imports, configure package discovery in pyproject.toml, install with uv pip install -e ., and run tests against the package imports.
The key distinction is: the current project asks Python to find individual files through sys.path; a packaged project asks Python to find one installed package through the

environment.

22. Troubleshooting checklist

If ModuleNotFoundError occurs, confirm the filename and capitalization, print sys.path, check the command’s working directory, check whether the import is package-qualified, confirm pytest.ini is in the repository root, and verify that the package was installed into the environment being used.

If pytest reports collected 0 items, verify that the file has functions or methods beginning with test_. If an AttributeError occurs, compare the method called in the test with the actual class API. If a relative data path fails from another directory, remember that imports and working-directory paths are separate concerns.